

AIM

Write a program to demonstrate the concept of deadlock avoidance through Banker's Algorithm.

SOFTWARE REQUIRED

- A Linux operating system
- GCC (GNU Compiler Collection) for compiling C programs
- Terminal or command line interface

THEORY

Deadlock avoidance is a technique used by the operating system to ensure that a system never enters a deadlock state. Unlike deadlock prevention, which restricts how resources can be requested, deadlock avoidance requires the system to have advance information about the maximum resources each process may request during its execution.

The Banker's Algorithm, developed by Edsger Dijkstra, is a resource allocation and deadlock avoidance algorithm that tests for safety by simulating the allocation of predetermined maximum possible resources and then makes a decision whether allocation should be allowed to continue. The algorithm works with the following data structures, where n is the number of processes and m is the number of resource types:

- **Available:** A vector of length m indicating the number of available resources of each type.
- **Max:** An $n \times m$ matrix that defines the maximum demand of each process for each resource type.
- **Allocation:** An $n \times m$ matrix that defines the number of resources of each type currently allocated to each process.
- **Need:** An $n \times m$ matrix indicating the remaining resource need of each process, calculated as $\text{Need} = \text{Max} - \text{Allocation}$.

The Safety Algorithm checks whether the system is in a safe state by attempting to find a sequence of processes such that each process's remaining need can be satisfied by the currently available resources plus the resources held by already finished processes. If such a sequence exists, the system is said to be in a safe state and the sequence is called a safe sequence; otherwise, the system may lead to a deadlock.

PROCEDURE

1. Create a C Program:

Write a C program that implements the Banker's Algorithm to check whether a given state of the system is safe, using the Allocation, Max, and Available data for a set of processes.

2.

```
3.  #include <stdio.h>
4.
5.  int main() {
6.      int n, m;
7.      int alloc[10][10], maxNeed[10][10], need[10][10];
8.      int avail[10], finish[10] = {0}, safeSeq[10];
9.
10.     printf("Enter number of processes and resources: ");
11.     scanf("%d %d", &n, &m);
12.
13.     printf("Enter Allocation Matrix:\n");
14.     for (int i = 0; i < n; i++)
15.         for (int j = 0; j < m; j++)
16.             scanf("%d", &alloc[i][j]);
17.
18.     printf("Enter Maximum Matrix:\n");
19.     for (int i = 0; i < n; i++)
20.         for (int j = 0; j < m; j++) {
21.             scanf("%d", &maxNeed[i][j]);
22.             need[i][j] = maxNeed[i][j] - alloc[i][j];
23.         }
24.     printf("Enter Available Resources:\n");
25.     for (int j = 0; j < m; j++)
26.         scanf("%d", &avail[j]);
27.     int count = 0;
28.     while (count < n) {
29.         int progressMade = 0;    // did any process finish in this pass?
30.         for (int i = 0; i < n; i++) {
```

```
31.         if (finish[i]) continue;
32.         int canRun = 1;
33.         for (int j = 0; j < m; j++) {
34.             if (need[i][j] > avail[j]) {
35.                 canRun = 0;
36.                 break;
37.             }
38.         }
39.         if (canRun) {
40.             for (int k = 0; k < m; k++)
41.                 avail[k] += alloc[i][k];
42.
43.             safeSeq[count++] = i;
44.             finish[i] = 1;
45.             progressMade = 1;
46.         }
47.     }
48.     if (!progressMade) break;    // stuck -> nothing more can run
49. }
50. if (count == n) {
51.     printf("Safe State\nSafe Sequence: ");
52.     for (int i = 0; i < n; i++)
53.         printf("P%d ", safeSeq[i]);
54.     printf("\n");
55. } else {
56.     printf("Unsafe State - Deadlock may occur\n");
57. }
58.
59. return 0;
```

60. Compile the Program:

OUTPUT

```
Enter number of processes and resources: 5 3
```

```
Enter Allocation Matrix:
```

```
0  1  0
2  0  0
3  0  2
2  1  1
0  0  2
```

```
Enter Maximum Matrix:
```

```
7  5  3
3  2  2
9  0  2
2  2  2
4  3  3
```

```
Enter Available Resources:
```

```
3  3  2
```

```
Safe State
```

```
Safe Sequence: P1 P3 P4 P0 P2
```

Upon executing the program, the console should display:

System is in a SAFE state.

Safe sequence: P1 -> P3 -> P4 -> P0 -> P2

CONCLUSION

This experiment successfully demonstrated the concept of deadlock avoidance using the Banker's Algorithm. By computing the Need matrix from the Allocation and Maximum matrices and applying the Safety Algorithm against the Available resources, the program was able to determine that the system was in a safe state and produce a valid safe sequence of process execution. This illustrates how an operating system can proactively avoid deadlock by granting resource requests only when the resulting state remains safe.